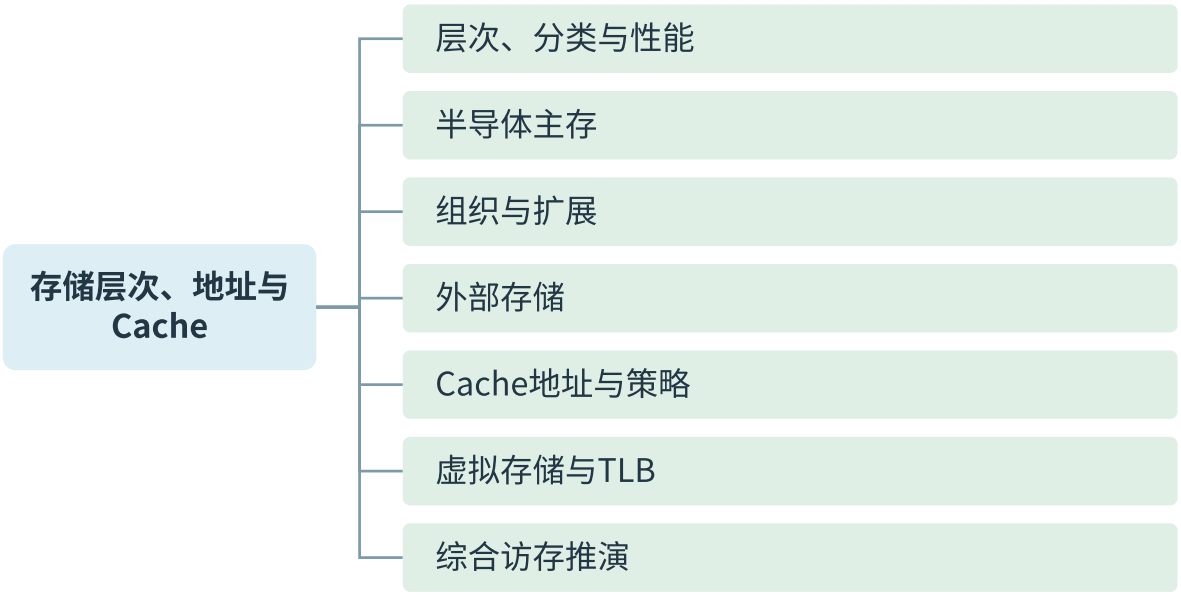
存储层次、地址与 Cache 3

章节脉络



本章围绕“层次、分类与性能 → 半导体主存 → 组织与扩展 → 外部存储 → Cache 地址与策略 → 虚拟存储与 TLB → 综合访存推演”展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

课程连接·分层连接操作系统

Cache 利用短期访问局部性，虚拟存储还承担地址空间、隔离和换页。两者都能体现层次思想，但缓存对象与缺失处理不同：Cache 常缓存数据块，TLB 缓存映射，页表记录虚页到页框关系。

3.1 为什么一个程序同时使用几层存储器

定义 3.1.1 (概念与适用对象)

寄存器、Cache、主存和外存的速度、容量及单位成本不同。寄存器紧贴运算部件、容量小；Cache 保存近期可能使用的主存块；主存容纳正在运行的程序和数据；外存提供较大且通常非易失的空间。层次化存储利用程序访问的局部性，使大量访问落在较快的层中，同时保留较大容量。

随机存取指访问时间基本不随单元位置改变，不等于“随机挑一个位置”。顺序存取要按顺序经过前面的内容；直接存取先定位一个区域，再在区域内访问，例如磁盘。相联存储器按内容匹配，TLB 的相联查找就是常见用途。

存取时间是完成一次读或写所需的时间；存取周期是连续两次访问之间允许的最短间隔。DRAM 读后需要恢复等操作，存取周期可能大于单纯读出延迟。计算连续访问的总时间时，不能只拿一次存取时间乘访问次数。

容量要写成“单元数 × 每单元位数”，再结合编址单位换算。2^{20} 个 8 位单元为 1 MiB；本讲义在芯片容量、地址空间、Cache 中使用二进制单位，主频和传输率中的 M、G 按十进制。题源沿用 KB、MB 时，根据地址幂次说明实际口径。

3.2 SRAM、DRAM 与 Flash 保存信息的方式

最小反例·动态不等于访问变化

最小反例：一个程序只反复读同一 DRAM 单元，芯片仍需刷新。动态描述电荷保持机制，而非程序写入频度；随机描述按地址选取能力，也不意味着无需刷新。把这两条分类轴分开。

SRAM 用稳定电路状态保存位，只要供电就不需周期刷新，通常速度快、集成密度相对低，适合 Cache。DRAM 用电容电荷表示位，电荷会泄漏，需要刷新；读出还伴随感知、放大和恢复。它的高密度使其适合主存。

“动态” 说的是保存机制，不是程序是否经常改写数据；“随机” 说的是访问方式，不是是否需要刷新。SRAM、DRAM 都是易失性存储器，掉电后不能保持原信息。

Flash 是非易失性半导体存储器。读、编程和擦除的粒度与代价不同，通常不能像 SRAM 那样直接覆盖写入任意旧位；例如 NAND Flash 常按页读写、按块擦除。SSD 控制器借助映射、垃圾回收和磨损均衡，把复杂的物理操作组织成主机看到的逻辑块访问。掉电保持不等于写入次数无限，也不等于写延迟与读延迟相同。

比较存储器时，要固定同一维度。SRAM 与 DRAM 在“能否随机访问”上相同，在“怎样保持一位、是否刷新、速度与密度”上不同；DRAM 与 Flash 在“是否掉电保持”上不同；Cache 与主存则首先是系统中的层次角色，常分别由 SRAM 和 DRAM 实现，而角色的划分与器件技术的划分不是一回事。芯片题再把器件组织写成单元数 $\times$ 位宽：一片 $4\text{M}\times 8$ 的容量是 4 MiB，不能因它用于主存，就把 “地址线、数据线、存储单元数” 当成同一个数。

3.3 地址引脚不总等于地址位数

SRAM 的 $2^k \times w$ 表示有 2^k 个可选单元，每次选中输出 w 位，通常需 k 根地址线和 w 根数据线，另有片选和读写等控制线。DRAM 常将行地址、列地址分时送入同一组地址引脚，由行、列选通信号分别锁存，因此逻辑地址位数不能直接作为地址引脚数。

3.3.1 例 3-1　DRAM 的复用地址怎样计数

例 3.3.1 (配套例题)

【配套例题】某容量为 256 MB 的存储器由若干 $4\text{M}\times 8$ 位的 DRAM 芯片构成，该 DRAM 芯片的地址引脚和数据引脚总数是（　）。

A. 19　B. 22　C. 30　D. 36

解析。$4\text{M}=2^{22}$，芯片内部需要 22 位单元地址。按题目采用的行列各 11 位、地址复用模型，需 11 根地址引脚；数据宽 8 位，需要 8 根数据引脚，共 19 根，选 **A**。题目给出的整机 256 MB 并不改变单片引脚数。若问组成整机需要多少片，则另算 $256/4=64$ 片。

实际 DRAM 还可能含多个 Bank，地址、命令及引脚结构要按题目给出的组织判断。刷新按行进行，不是每保存一个位就分别发一次刷新。集中刷新把若干刷新操作连在一起，期间可能有一段较长的时间不能正常访问；分散刷新把一次正常存取和一次刷新组合到扩大的存取周期内，不形成集中刷新的长间隙，但会降低正常访存速率。异步刷新在保持时间内把各行刷新均匀分散，定期插入短的刷新周期，其间仍可进行正常访问。若有 R 行、保持时间为 T_r，把每行各刷新一次并均匀分布，平均间隔为 T_r/R；还须检查每次刷新占用多久。

3.3.2 MAR 的位数由地址空间决定

最小反例·已安装容量不决定 MAR

最小反例：机器给出 64MiB 字节地址空间，只安装 32MiB 内存。MAR 仍需容纳 26 位地址，而非按已安装容量只取 25 位。芯片单元地址、可编址空间、已安装容量是三个问题。

MAR 需容纳可由该机器给出的主存地址。它与 MDR 的位数没有 “必须相等” 的关系。MDR 反映某次数据传送的位宽，MAR 反映地址编码位宽。

3.3.3 例 3-2　已安装容量不是全部可编址空间

例 3.3.2 (配套例题)

【配套例题】某计算机存储器按字节编址，主存地址空间大小为 64 MB，现用 $4\text{M}\times 8$ 位的 RAM 芯片组成 32 MB 的主存储器，则存储器地址寄存器 MAR 的位数至少是（　）。

A. 22 位　B. 23 位　C. 25 位　D. 26 位

解析。题目问覆盖 64 MB 地址空间需要多少位，而不是按已安装的 32 MB 去算。$64\text{MB}=2^{26}\text{B}$，每个字节一个地址，所以 MAR 至少 26 位，选 **D**。芯片地址位数 22、已安装容量地址位数 25 都不能回答这一问题。

3.4 单体多字和多体交叉怎样提高带宽

单体多字存储器扩宽一次读写的数据量，一个存储单元包含多个机器字。顺序使用相邻字时，一次访问可以提供多个字；若频繁跳转，取来的其他字未必有用。它主要改善带宽，不能仅凭 “一次取多个字” 就当成解决地址空间不足的方案。

多体存储器则包含能相对独立工作的存储模块。高位交叉（顺序编址）用地址高位选模块，各模块保存连续的一段地址；低位交叉用低位选模块，连续字地址轮流落到不同模块。对 m 个模块且按字编址的常规模型：

$$\text{模块号}=\text{字地址 mod } m,\qquad \text{模块内地址}=\left\lfloor\frac{\text{字地址}}{m}\right\rfloor.$$

若给的是字节地址，每字 w B，且各模块每次存取一字，通常应先由 $\lfloor A/w\rfloor$ 求出字地址，再据此分模块；最低的字节偏移不参与模块编号。多体交叉与单体多字可以结合，但两者不是同一种机制。

设单模块存取周期为 T，连续启动相邻模块的间隔为 r，模块数 m。连续顺序访问不冲突的条件是 $mr\ge T$。若第一字在 T 后到达，后续每隔 r 得到一个字，读 n 个字的时间为 $T+(n-1)r$。第一字延迟仍为 T，提高的是连续吞吐率。

3.4.1 例 3-3　同一模块不等于立即发生冲突

例 3.4.1 (配套例题)

【配套例题】某计算机使用 4 体交叉编址存储器，假定在存储器总线上出现的主存地址（十进制）序列为 8005、8006、8007、8008、8001、8002、8003、8004、8000，则可能发生访存冲突的地址对是（　）。

A. 8004 和 8008　B. 8002 和 8007　C. 8001 和 8008　D. 8000

和 8004

解析。先求地址模 4：1、2、3、0、1、2、3、0、0。最后两次 8004、8000 连续访问模块 0，模块可能尚未结束前次存取，所以选 **D**。8008 与 8004 也在模块 0，却隔了四个访问位置，不能只凭同体就忽略时间间隔。这里沿题目的低位交叉约定解释 “可能冲突”，具体是否冲突仍由模块周期与请求间隔决定。

不变量·位扩展共享片选

同一地址上的多片芯片分担不同数据位，因此片选和片内地址应同步；不同地址组才由高位译码区分。先画组，再连数据线，能避免把位扩展误画成容量分区。

3.5 主存芯片怎样连到 CPU

位扩展把若干芯片并联在同一地址上，各提供数据的一部分；字扩展让若干组芯片承担不同地址范围，用片选区分。组成 $M \times W$ 的存储器，单片容量 $m \times w$，且两个方向都能整除时，位方向需要 W/w 片，字方向需要 M/m 组，总片数为两者乘积。

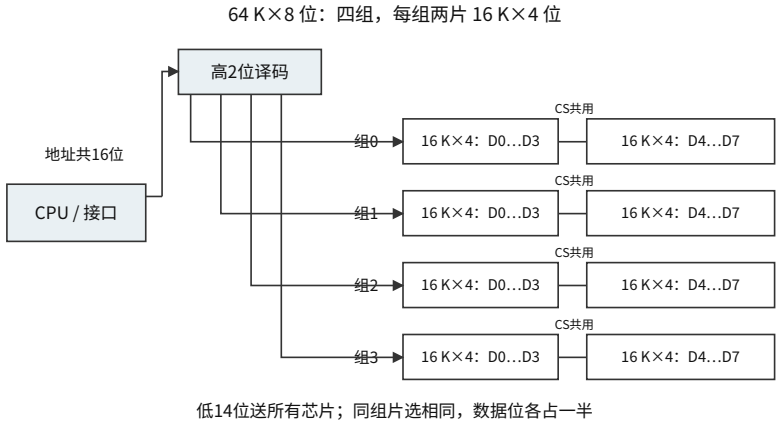


Figure 3.1: 两片并联扩位，四组译码扩字；同组选中信号相同

图中每片 16K × 4 位，同行两片并成 16K × 8 位，四组组成 64K × 8 位。低 14 位送给所有芯片选择片内单元，高 2 位经 2–4 译码器选一组。位扩展增加数据宽度，字扩展增加可选单元数，两者所用连线不同。

3.5.1 例 3-4　ROM 与 RAM 分区要分别计算

例 3.5.1 (配套例题)

【配套例题】某计算机主存容量为 64 KB，其中 ROM 区为 4 KB，其余为 RAM 区，按字节编址。现要用 2K × 8 位的 ROM 芯片和 4K × 4 位的 RAM 芯片来设计该存储器，则需要上述规格的 ROM 芯片数和 RAM 芯片数分别是（　）。

A. 1、15　　B. 2、15　　C. 1、30　　D. 2、30

解析。 ROM 单片 2 KB 且已有 8 位宽，需 $4/2 = 2$ 片。RAM 总量 60 KB；单片虽有 4 K 单元，每单元只有 4 位，要两片并成 4K × 8 位的一组，再用 $60/4 = 15$ 组，共 30 片，选 **D**。只算 60/4 会漏掉位扩展。

确定地址范围时，先确定编址单位，再拼接片选地址和片内地址。首地址由片选编码后接全 0 构成，末地址由片选编码后接全 1 构成。例如按字编址，32 K×16 位芯片的片内字地址为 15 位；若高 3 位片选为 111，则字地址范围为 `38000H` 至 `3FFFFH`。若改按字节编址，每个字占 2 B，片内字节范围需 16 位，范围随之改变，不能沿用同一答案。

地址线的编号方式同样要从题干读。通常本书令 A0 为最低位；有的题明确写 A0 为最高位，此时 4 K 片内地址使用最后 12 根低位地址线，片选应取其上方的线。下标数字不是判断高低位的可靠依据。

错项排除·平均旋转等半圈

转速 r 转每分意味着一圈 60/r 秒；均匀随机角位置的平均等待为半圈，传一扇区另按一圈中该扇区比例计算。SSD 没有这段机械旋转过程，不能给它机械套用半圈等待。

3.6 磁盘与 SSD 的一次访问花在哪些时间上

磁盘由盘面、磁道和扇区组织信息。同一半径处各盘面的磁道组成柱面；选择盘面由磁头完成，寻道把磁头移动到相应磁道。旋转等待使目标扇区转到磁头下方，随后才传输数据。磁盘地址的典型几何字段是柱面号、磁头号、扇区号；磁头号已经指定盘面，不必再把 “盘面号” 另算一个定位维度。

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}.$$

只有题设要求时再计控制器或排队开销。转速为 r 转/分时，一圈为 60/r 秒；随机到达且目标角位置均匀时，平均旋转等待为半圈。传输一个扇区的时间只占一圈的相应部分，不能把等待的半圈再算进传输时间。

3.6.1 例 3-5　7200 转每分不是 7200 转每秒

例 3.6.1 (配套例题)

【配套例题】若磁盘转速为 7200 转/分，平均寻道时间为 8 ms，每个磁道包含 1000 个扇区，则访问一个扇区的平均存取时间大约是（　）。

A. 8.1 ms　B. 12.2 ms　C. 16.3 ms　D. 20.5 ms

解析。一圈 $60/7200$ s ≈ 8.333 ms，平均旋转等待约 4.167 ms；一扇区传输约 $8.333/1000 = 0.008333$ ms。总时间约 12.175 ms，选 **B**。把一整圈当成平均等待，会得到另一数量级的选项。

SSD 没有机械寻道和旋转等待，但仍有接口传输、闪存访问、控制器与写放大等开销。因此不能照搬磁盘的平均旋转等待公式。RAID 把多个盘组织起来：条带化提升并行访问能力，镜像提供冗余，奇偶校验可支持某些盘故障后的恢复。RAID0 没有数据冗余；即便采用冗余，也不等于可以代替备份：误删的内容仍可能被同步写到冗余盘。

3.7 Cache 命中率怎样变成时间

时间局部性指近期访问过的信息可能很快再访问，例如循环累加器；空间局部性指相邻位置可能很快被访问，例如顺序扫描数组。Cache 按块调入，因此顺序数组中一次缺失可能为随后几个元素准备数据。

命中率 $h = N_{\text{hit}}/(N_{\text{hit}} + N_{\text{miss}})$。平均访问时间必须先明确 “未命中” 指的是哪一段时间。若先查 Cache 花 t_c，未命中后再用 t_m 完成取块，则

$$\text{AMAT} = t_c + (1 - h)t_m.$$

若题目给出的 “未命中访问时间” 已经包含查 Cache 的时间，记为 t_{miss}，则写为 $ht_c + (1-h)t_{\text{miss}}$。两式只是对同一事件的不同分解，第

二式中不能再额外加一次 t_c。多级 Cache 则应区别局部缺失率与相对 CPU 总访问的全局缺失率。

最小反例·循环轮数不是访问次数

最小反例：一条 a[k]=a[k]+32 每轮读一次又写一次，因此四元素八次数据访问。空 Cache 顺序扫描时一块首次读缺失，后续写命中。分母错用四轮，就把正确 12.5% 误算成 25%。

3.7.1 例 3-6　读后写的数组访问怎样计缺失率

例 3.7.1 (配套例题)

【配套例题】有如下 C 语言程序段：

```
for (k = 0; k < 1000; k++)
    a[k] = a[k] + 32;
```

若数组 a 和变量 k 均为 int 型，int 型数据占 4 B，数据 Cache 采用直接映射方式，数据区大小为 1 KB、块大小为 16 B，该程序段执行前 Cache 为空，则该程序段执行过程中访问数组 a 的 Cache 缺失率约为（　）。

A. 1.25%　　B. 2.5%　　C. 12.5%　　D. 25%

解析。每块容纳 4 个 int。按顺序扫描，每块第一次读缺失，其余 3 次读命中；第一次读已把块取入，随后对应的写也命中。每 4 个元素共 8 次数组访问、1 次缺失，因此约为 12.5%，选 **C**。若数组首地址未按块对齐，块数可能多 1，不影响本题 “约为” 的结论。不能把循环 1000 轮误当作只访问数组 1000 次。

3.7.2 一次缺失损失要不要反复付启动代价

若每次取 8 B 需发送地址 1 周期、准备 8 周期、传输 1 周期，64 B 块包含 8 次独立访问，总计 $8(1+8+1)=80$ 周期。若支持突发，只发一次首地址，等待首数据 8 周期后连续传 8 次，则共 $1+8+8=17$ 周期。是否支持突发会改变这里的计算结果，详见第 6 章；不能只看总线同为 64 位就给出同一个时间。

3.8 地址拆分决定块能放在哪里

先由数据容量 C 和块大小 B 求行数 $L=C/B$。若每组 r 行，则组数 $G=L/r$。按字节编址且 B、G 为 2 的幂时，块内偏移为 $b=\log_2 B$ 位，组号为 $g=\log_2 G$ 位，剩下高位为标记。若 Cache 总容量已经包含标记等附加位，这个总量就不能再当作 C。

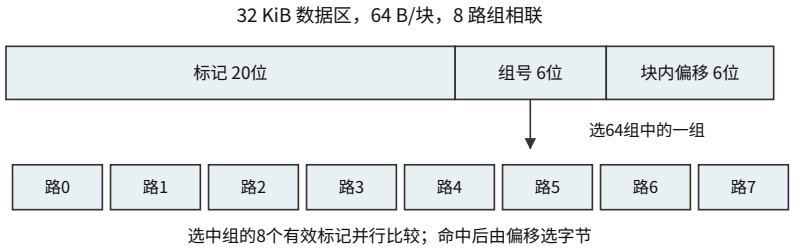


Figure 3.2: 先去块内偏移，再用块号确定组和标记

常规低位选组的规则为：主存块号 $q=\lfloor A/B \rfloor$，组号 $q \bmod G$，标

记 $\lfloor q/G \rfloor$。直接映射是 r=1，全相联是 G=1；r 路组相联中的 “路数” 是每组行数，不是组数。

在地址位数、数据容量 C 和块大小 B 都相同的条件下，三种映射可放在同一框架比较。表中 C 为数据区容量，与 B 采用相同单位；块大小和组数按 2 的幂组织，字段位数才可直接写成以 2 为底的对数。

映射	每块可放的位置	地址中的组选字段	一次查找的比较范围
直接映射	唯一一行	$\log_2(C/B)$ 位	选中行的一个标记
r 路组相联	唯一一组内的 r 行	$\log_2(C/(Br))$ 位	该组 r 个有效标记
全相联	全 Cache 任一行	0 位	全部行的有效标记

同一过程的其余状态（按首列对应）：

映射	满时怎样选牺牲行
直接映射	无行间选择
r 路组相联	仅在该组内替换
全相联	在全 Cache 内替换

最小反例·命中会改变 LRU

最小反例：两路一组访问 A、B、A、C。LRU 在第三次命中 A 后淘汰 B，FIFO 仍按装入次序淘汰 A。只记录装入顺序无法模拟 LRU，命中也必须更新最近使用状态。

提高相联度时，块内偏移不变，组选字段变短，标记相应变长；比较器和替换状态的成本也可能增加。这样做能减少某些冲突缺失，却不消除首次访问的强制缺失，也不能让有限容量容纳任意大的工作集。映射决定候选位置，替换决定满组时选谁，写策略决定何时更新下一层，应分别求解。

访问时用组号选一组，并行比较该组各行的有效标记。组号已经用于选组，不需要再保存到每行标记中；块内偏移只负责从已找到的块中选字节。

3.8.1 例 3-7　8 路组相联需要几个比较器

例 3.8.1 (配套例题)

【配套例题】若计算机主存地址为 32 位，按字节编址，某 Cache 的数据区容量为 32 KB，主存块大小为 64 B，采用 8 路组相联映射方式，该 Cache 中比较器的个数和位数分别为（　）。

A. 8，20　　B. 8，23　　C. 64，20　　D. 64，23

解析。行数 $2^{15}/2^6 = 2^9$，组数 $2^9/8 = 2^6$。偏移 6 位、组号 6 位，标记 $32-6-6 = 20$ 位。一次只在选中的一组内比较 8 个标记，因此 8 个 20 位比较器，选 **A**。组数为 64，并不表示同一次访问就要比较 64 个组。

3.9 LRU 更新的是访问次序，不只是装入次序

当目标组有空行时先用空行；满了才按替换算法决定牺牲哪一行。直接映射每个块只有一个位置，不需要在多行间选择。FIFO 记录装入次序，命中一般不改变队列；LRU 记录最近访问次序，命中也要更新；随机替换不跟踪精确最近次序。

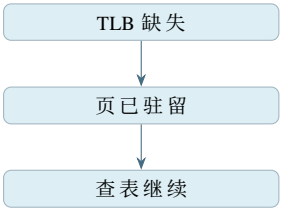
下面以两路的一个组解释 LRU 状态，顺序从最近使用到最久未使用。开始为空，访问 A 后为 [A]，访问 B 后为 [B,A]，再次访问 A 后为 [A,B]。此时访问 C，应淘汰 B，变为 [C,A]。若按 FIFO，则第三次访问 A 不改变装入队列，访问 C 会淘汰 A。这组状态足以说明两算法的分歧。

执行替换题的表格应至少记录本次地址、块号、组号、命中与否，以及该组更新后的状态。替换仅发生在对应组内，不能从其他组挑一个

“全 Cache 最久未用” 的块。若题干改用高位或特殊函数选组，先按题干给出的映射分组；LRU 本身不负责决定组号。

最小反例·映射没缓存不等于没驻留

最小反例：TLB 未命中，但页表存在位为 1。查表后即可得到页框号，无需从磁盘调页。TLB 缺失只说明映射未缓存，缺页才说明合法目标页当前不驻留。



TLB 未命中且存在位为 1，不发生缺页。

3.10 写命中与写缺失是两个独立问题

写命中时，直写同时更新 Cache 和下一层；回写只改 Cache 并置修改位，替换脏块时才写回。直写常配写缓冲，以减少等待，但缓冲满时仍可能阻塞。回写时主存内容暂时旧于 Cache，这是允许的受控状态，不能凭 “二者立即不同” 就断定设计有错。

写缺失时，写分配先调入块再更新 Cache，非写分配直接写下一层而不把该块装入 Cache。回写常配写分配、直写常配非写分配，这是常用组合，不是逻辑上仅有的两种组合。

3.10.1 例 3-8　Cache 总容量还包括哪些位

例 3.10.1 (配套例题)

【配套例题】假定主存地址为 32 位，按字节编址，主存和 Cache 之间采用直接映射方式，主存块大小为 4 个字，每字 32 位，采用回写方式，则能存放 4 K 字数据的 Cache 的总容量的位数至少是（　）。

A. 146 K　　B. 147 K　　C. 148 K　　D. 158 K

解析。一块 $4 \times 4 = 16$ B，块内偏移 4 位。4 K 字数据共 16 KB，Cache 有 $4096/4 = 1024 = 2^{10}$ 行，行号 10 位，标记 18 位。每行数据 128 位，还需有效位 1 和修改位 1，共 $128 + 18 + 1 + 1 = 148$ 位；总计 $1024 \times 148 = 148$ K 位，选 **C**。直接映射无需 LRU 替换位。本题 “至少” 按这些必要字段计算，不额外猜测 ECC 等未给出的实现。

多个 CPU 各有 Cache 时，一个核的写入还涉及其他副本怎样失效或更新；DMA 与 Cache 也可能有一致性处理。单核回写正确不等于多核副本会自动一致。协议细节超出本章基本模型，此处保留 “需要一致性机制” 的边界，不把全写法当作解决任意多副本问题的证明。

3.11 虚拟存储器怎样让程序看到地址空间

虚拟存储器把进程使用的虚拟地址映射到物理地址，允许部分内容暂存外存，也支持隔离与保护。它不是 “用硬盘代替所有内存”：CPU 最终仍需要可访问的物理存储，缺页时还要调入并等待。

操作系统建立、维护页表等映射信息，决定分配、换入换出及保护；运行时 MMU 根据这些信息实施地址转换并检查权限。把 “映射由 OS 管理” 与 “每次访存都调用 OS 软件转换” 混为一谈，会误判硬件路径。正常的 TLB 命中不需要逐次陷入内核。

3.12 页表与 TLB：地址偏移为何原样保留

页式系统把虚拟地址分为虚页号 VPN 和页内偏移 d；页表查得物理页框号 PPN 后，物理地址为 PPN $\times P + d$，P 是页大小。因为页和页框大小相同，页内位置不变。虚拟地址位数和物理地址位数可以不同，改变的是页号部分。

TLB 是页表项的高速缓存。TLB 未命中表示缓存里没有本次所需的映射，不表示页面一定不在主存；随后可以查页表并填入 TLB。对于地址合法且映射已建立的页面，存在位为 0 表示尚未驻留，按题设触发缺页异常。非法地址、无效映射或权限不足的情况须另行检查，不能一律按可调页修复处理；保护异常也可能发生在页面已经在主存时。

不变量·写回与写分配独立

直写或回写回答命中时怎样更新下一层，分配或不分配回答缺失时是否调入块。常见搭配不是逻辑上的唯一搭配。读清题设组合，再决定缺失数、主存写次数和脏块处理。

3.12.1 TLB、页表与 Cache 各自缺少什么

结构	查找所依据的信息	保存的主要对象	未命中或无效后继续做什么
TLB	虚页号；组相联时再拆组号和标记	虚页到物理页框的映射及权限等	查页表；此时还不能断定缺页
页表	虚页号，可能经过多级索引	页框号、存在位、权限及管理状态等	地址合法且未驻留时按题设缺页；非法映射与保护失败另行处理
物理地址访问的 Cache	物理块号；再拆组号和标记	主存数据块或指令块的副本	访问下一存储层，按写策略调块或写出

因此，TLB 命中仍可能 Cache 缺失；TLB 未命中也可能页面已在主存。不能把三个结构的 “有效” 都当成 “原数据已经在 Cache”。TLB 表项存的是地址映射，不是一页完整数据；Cache 行的标记也不等同于物理页框号。

按 “先转换、后查物理 Cache” 的串行教学模型，一次普通数据读可这样展开：先由虚拟地址分出 VPN 与页内偏移；用 VPN 查 TLB，未命中则走题设页表层数；取得有效页框号后拼出物理地址；再由物理地址分 Cache 标记、组号、块内偏移，决定从 Cache 还是下一层取得数据。若中间缺页，先处理并重试指令，不把尚未形成的物理地址继续送去查数据 Cache。计算次数或时间时，就沿这条路径逐项累计；页表访问是否又经过 Cache、TLB 与 Cache 能否重叠，均须服从题设。

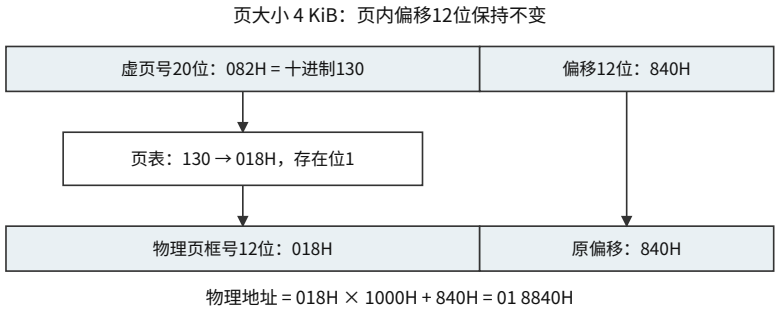


Figure 3.3: 虚拟地址先查映射，再用页框号与原偏移形成物理地址

3.12.2 例 3-9　页号的十六进制不能直接当表中十进制

例 3.12.1 (配套例题)

【配套例题】某计算机主存地址为 24 位，采用分页虚拟存储管理方式，虚拟地址空间大小为 4 GB，页大小为 4 KB，按字节编址。某进程的页表部分内容如下。

虚页号	实页号（页框号）	存在位
82	024H	0
…	…	…
129	180H	1
130	018H	1

当 CPU 访问虚拟地址 0008 2840H 时，虚—实地址转换的结果是（ ）。

A. 得到主存地址 02 4840H　B. 得到主存地址 18 0840H
C. 得到主存地址 01 8840H　D. 检测到缺页异常

解析。页大小 2^{12} B，低 12 位是偏移 `840H`，虚页号 `82H` 等于十进制 130，并非表中第一行的十进制 82。查到页框 `018H` 且存在位 1，物理地址为 `018H×1000H+840H=018840H`，选 **C**。计算前逐列辨明进制，可避免把本题误判为缺页。

3.12.3 例 3-10　TLB 标记不包含页内偏移

例 3.12.2 (配套例题)

【配套例题】某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为 32 位，主存地址为 30 位，页大小为 1 KB。若 TLB 共有 32 个表项，采用 4 路组相联映射方式，则 TLB 表项中标记字段的位数至少是（ ）。

A. 17　B. 18　C. 19　D. 20

解析。偏移 10 位，VPN 为 22 位。TLB 共有 8 组，选组用 VPN 的 3 位，标记为 $22-3=19$ 位，选 **C**。物理地址 30 位决定页框号位数，不决定用来匹配虚页的标记位数；也不能把 Cache 块偏移的位数套用到 TLB 标记上。

计算含 TLB 的平均时间时，应画出题设路径。例如，设访问串行进行，TLB 命中率为 h，TLB 耗时 t_T，单级页表和主存访问各耗时 t_M，则不计缺页时的平均时间为 $h(t_T+t_M)+(1-h)(t_T+2t_M)$。多级页表、页表项缓存、TLB 与 Cache 并行查找都会改变路径。缺页还包括外存和操作系统处理，通常远大于一次普通主存访问。

3.13 段式与段页式地址为什么需要界限检查

不变量·段界限先约束偏移

段式地址包含段号与段内偏移，应先核段表和偏移是否在界限内，再形成物理地址或进入分页。扩大基址不能修复越界偏移；非法地址与合法页未驻留也不能混成同一种缺页。

段式系统按代码、数据等逻辑单位划分不等长段。逻辑地址含段号和段内偏移，段表记录段基址、段长及权限。若段长表示可用字节数 L，必须先验证 $0 \le d < L$，再计算物理地址 base $+ d$；当 d=L 时已经越界，不能把末地址写成 base+L。

段页式先按段号找到该段页表，再用段内页号查页框，最后拼接页内偏移。段表基址、页表基址和最终数据地址不是同一个量。若段本身有界限，页号合法仍不保证最后一页的所有偏移均合法。

例如某段基址 10000 B、段长 3000 B，偏移 2999 合法、物理地址 12999 B；偏移 3000 非法。若段页式采用 1 KB 页，该段的逻辑长度仍为 3000 B，最后一个页号 2 中的偏移 952 对应段内 3000，应被界限检查拒绝，不能因页框还有空间就允许访问。

3.14 综合访存：数组次序、TLB 替换与 Cache 查找

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

3.14.1 例 3-11　交换内外层循环，命中率一定降低吗

例 3.14.1 (配套例题)

【配套例题】改编（两个赋值式统一为常量 10）已知计算机 M 字长为 32 位，按字节编址，采用请求调页策略的虚拟存储管理方式，虚拟地址 32 位，页面大小为 4 KB；数据 Cache 采用 4 路组相联映射方式，数据区大小为 8 KB，主存块大小为 32 B。现有 C 语言程序段如下：

```
int a[24][64];
/* ... */
for (i = 0; i < 24; i++)
    for (j = 0; j < 64; j++) a[i][j] = 10;
```

已知二维数组 a 按行优先存放，在虚拟地址空间中分配的起始地址为 0042 2000H，`sizeof(int)=4`。假定在 M 上执行上述程序段之前数组 a 不在主存，且在该程序执行过程中不会发生页面置换。请回答下列问题。

（1）数组分布在几个页面中？对于数组 a 的访问，会发生几次缺页异常？页故障地址各是什么？

（2）不考虑变量 i 和 j，该程序段的数据访问是否具有时间局部性？为什么？

（3）计算机 M 的虚拟地址（A31～A0）中哪几位用作块内地址？哪几位用作 Cache 组号？a[1][0] 的虚拟地址是多少？其所在主存块对应的 Cache 组号是多少？

（4）数组 a 占用多少主存块？假设上述程序段执行过程中数组 a 的访问不会和其他数据发生 Cache 访问冲突，则数组 a 的 Cache 命中率是多少？若将循环中 i 和 j 的次序按如下方式调换，则数组 a 的 Cache 命中率又是多少？

```
for (j = 0; j < 64; j++)
    for (i = 0; i < 24; i++) a[i][j] = 10;
```

解析。先把数组元素下标化为字节地址，再分别按页和 Cache 块划分。按行优先，每行 64 个 int，共 256 B，因此

$$\mathrm{addr}(a[i][j]) = \texttt{00422000H} + (64i + j) \times 4.$$

本题参考解按写分配模型处理首次写缺失：写一个尚不在 Cache 中的块时先调入该块。后续命中率均采用这一约定；若明确改为非写分配，结论须重算。数组初始不在主存，因此不能把数组块预先算作有效命中。

（1）数组大小为 $24 \times 64 \times 4 = 6144$ B，即 6 KiB。起始地址低 12 位为 0，恰在 4 KiB 页边界；前 16 行占第一页 4096 B，后 8 行占第二页

2048 B，所以共 2 页。

第一次执行 a[0][0] 时缺页，故障地址为 0042 2000H；第一次执行 a[16][0] 时进入下一页，故障地址为 0042 3000H。无页面置换，每页首次调入后保留，共 2 次缺页。6 KiB 除以 4 KiB 后向上取整在这里成立，是因为首地址页对齐；若数组从页中部开始，必须把首地址页内偏移也计入。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

（2）以数组元素为数据对象，各元素只被写一次，没有重复访问同一元素，因此没有时间局部性。按行连续访问相邻元素，具有空间局部性。同一 Cache 块中相邻元素的后续访问能够命中，正是按块调入利用空间局部性的结果；不应据此认为同一元素 “又访问了一次”。

（3）块为 2^5 B，块内地址为 A4～A0。Cache 行数 $8192/32 = 256$，4 路共有 64 组，组号 6 位，对应 A10～A5。虽然映射最终作用于物理块，但这些位都在低 12 位页内偏移内，地址转换不改变它们，因此可以直接从虚拟地址取出本题组号。

a[1][0] 在起点之后 256 B，即 0042 2100H。低 11 位中的组号为 `(2100H>>5) mod 64=8`，二进制 001000。不要把 “第 1 行” 当成只跨过一个 int。

（4）每块 32 B 可容纳 8 个 int，起点块对齐，总块数 $6144/32 = 192$。按行访问时，每块第一次写缺失，随后 7 次写命中，共 1536 次写、192 次缺失，命中率为

$$\frac{1536 - 192}{1536} = \frac{7}{8} = 87.5\%.$$

交换循环后，每个 j 先跨 24 行访问，同一列相邻两次地址相差 256 B，即相差 8 个块，组号也按模 64 增加 8。固定 j 时，24 个块轮流落在 8 个组中，每组恰好 3 块，**4 路容得下这 3 块**。因此不能仅凭 “按列走” 就断言块被赶走。

例如 j=0 时，行号 0、8、16 的块都映射到组 0，分别占据该组 3 路；第 4 路仍空。j=1 至 7 仍访问这 24 个已装入块，均命中。j=8 首次使用下一批 24 块，其组号在原来基础上加 1，与前批分属不同组；以后同理。全数组 192 块按映射可以同时驻留，仍只有 192 次首次缺失，命中率仍为 87.5%。

这个结论同时依赖数组大小、起始对齐、组数、路数和无其他数据冲突这些条件。不能只凭 “数组 6 KiB 小于 Cache8 KiB” 就下结论，因为总容量装得下，仍可能在少数组内发生冲突。

关键变式（原创）。若仅把 Cache 改成 4 KiB、2 路，保留 64 组和 32 B 块，则每个相关组仍轮流访问 3 块，却只有 2 路。按列遍历、LRU 替换时，三块循环会互相淘汰，下一列也难以复用，数组写访问全部缺失；按行遍历仍能在同块的 8 次访问中命中 7 次。决定解法的条件是每组能容纳的块数，而不是 “换循环必然降低命中率” 这类笼统说法。

3.14.2 例 3-12　TLB 的组选、标记与连续替换

例 3.14.2 (配套例题)

【配套例题】假设计算机 M 的主存地址为 24 位，按字节编址；采用分页存储管理方式，虚拟地址为 30 位，页大小为 4 KB；TLB 采用 2 路组相联方式和 LRU 替换策略，共 8 组。请回答下列问题。

（1）虚拟地址中哪几位表示虚页号？哪几位表示页内地址？

（2）已知访问 TLB 时虚页号高位部分用作 TLB 标记，低位部

分用作 TLB 组号，M 的虚拟地址中哪几位是 TLB 标记？哪几位是 TLB 组号？

（3）假设 TLB 初始时为空，访问的虚页号依次为 10、12、16、7、26、4、12 和 20，在此过程中，哪一个虚页号对应的 TLB 表项被替换？说明理由。

（4）若将 M 中的虚拟地址位数增加到 32 位，则 TLB 表项的位数增加几位？

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

解析。这里替换的是地址映射表项，先按 VPN 选 TLB 组，再在所选组内更新 LRU；不依据物理页框号选组，也不对全部 16 项作一次全局 LRU。

（1）4 KB=2^{12} B，故虚拟地址 A11～A0 为页内偏移，A29～A12 共 18 位为 VPN。物理地址 24 位只决定页框号为 12 位，不能据此把虚页号也截成 12 位。

（2）8 组需 3 位组号，取 VPN 最低 3 位，即虚拟地址 A14～A12；余下 A29～A15 共 15 位为 TLB 标记。把字段写成位数核对：$15+3+12=30$。

（3）题中虚页号均为十进制。组号为 VPN mod 8，标记为 $\lfloor \mathrm{VPN}/8 \rfloor$。下表的状态从最久未使用到最近使用，只列本次选中的组。

次序	VPN	组号	标记
1	10	2	1
2	12	4	1
3	16	0	2
4	7	7	0
5	26	2	3
6	4	4	0
7	12	4	1

次序	VPN	组号	标记
8	20	4	2

同一过程的其余状态（按首列对应）：

次序	命中与否	本次访问后该组的 VPN 次序
1	未命中，空行装入	[10]
2	未命中，空行装入	[12]
3	未命中，空行装入	[16]
4	未命中，空行装入	[7]
5	未命中，第二路装入	[10,26]
6	未命中，第二路装入	[12,4]
7	命中，更新最近次序	[4,12]

次序	命中与否	本次访问后该组的 VPN 次序
8	未命中，淘汰 4	[12,20]

只有组 4 遇到第三个不同虚页。第 7 次命中 12 后，最久未用的是 4，所以访问 20 时替换 VPN=4 的表项。若只按装入次序排队，会错误淘汰 12；若只看全部 16 项里已用了几项，还会误以为未满就无需替换。

（4）页大小、组数和主存地址位数都不变。新增的 2 位虚拟地址属于 VPN 高位，全部进入 TLB 标记；页框号仍 12 位，故每个 TLB 表项增加 2 位。增加虚址位宽扩大了可区分的虚页范围，不会自动增加实际主存容量或 TLB 项数。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

3.14.3 例 3-13　TLB 未命中后，还要经过哪些判断

例 3.14.3 (配套例题)

典型示例（据王道《操作系统》§3.2.9 摘编、重排）。某系统按字节编址，虚拟地址为 14 位，物理地址为 12 位，页大小为 64 B。TLB 共 16 项，采用 4 路组相联；数据 Cache 使用物理地址，采用直接映射，共 16 行，每行 4 B。

按“先完成地址转换，再查数据 Cache”的串行模型分析。所有待访问的虚拟地址均合法，且允许读；页表存在位 $P=1$ 表示页面已在主存，$P=0$ 表示合法但尚未驻留，需要处理缺页。读 Cache 未命中时，从主存调入对应的 4 B 数据块。

以下给出本题所需的初始状态。表中页号、页框号、标记、Cache 行号和字节数据均用十六进制，后缀 H 表示十六进制；V 为有效位，“—”表示没有可用信息。TLB 的组号和路号均在 0 至 3 之间。

表 A　TLB 初始状态：每行对应一个表项

组号	路号	标记	有效位 V
0	0	03H	0
0	1	00H	0
0	2	09H	1
0	3	07H	1
3	0	07H	0
3	1	0AH	1
3	2	03H	1

组号	路号	标记	有效位 V
3	3	02H	0

同一过程的其余状态（按首列对应）：

组号	页框号
0	—
0	—
0	0DH
0	02H
3	—
3	34H
3	0DH

组号	页框号
3	—

表 B　页表中与本题有关的表项

虚页号	存在位 P	页框号
03H	1	02H
08H	0	—

表 C　Cache 初始状态：数据列按块内字节偏移排列

行号	标记	V	偏移 0
5H	0DH	1	36H
CH	02H	0	—

同一过程的其余状态（按首列对应）：

行号	偏移 1	偏移 2	偏移 3
5H	72H	F0H	1DH
CH	—	—	—

（1）写出查 TLB 时虚拟地址的字段划分，以及查 Cache 时物理地址的字段划分，并求各字段位数。

（2）分别读取虚拟地址 03D4H、00F1H、0229H 处的一个字节，依次说明 TLB 查找、必要的页表检查和 Cache 查找，指出最终从哪里取数，或在哪一步转入异常处理。能确定字节值时给出结果。

三次访问相互独立，均从上述初始状态开始。某次访问引起的回填不影响另两次；不推演缺页后的页框分配与替换，也不计算访问时间。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

解析。先分别拆分虚拟地址和物理地址，再沿实际访问路径检查各表项。

（1）两种地址的字段划分。页大小为 $64 = 2^6$ B，页内偏移占 6 位，虚页号 VPN 占 $14 - 6 = 8$ 位。TLB 有 $16/4 = 4$ 组，用 VPN 的低 2 位选组，其余 6 位作为标记。因此，从高位到低位，虚拟地址分为 **TLB 标记 6 位、组号 2 位、页内偏移 6 位**。

Cache 每行 4 B，块内偏移占 2 位；共 16 行，行号占 4 位，其余 $12 - 4 - 2 = 6$ 位作为标记。因此，物理地址分为 **Cache 标记 6 位、行号 4 位、块内偏移 2 位**。页内偏移和 Cache 块内偏移的长度不同，不能混用。

（2）三次独立的读访问。判断顺序如下。

- **查 TLB：**在选中组内，只有 $V = 1$ 且标记相同的表项才算命中。命中可取得页框号；未命中则查页表，此时还不能断定缺页。
- **必要时查页表：**本题已保证地址合法且有读权限。若 $P = 1$，取得页框号，并可回填 TLB；若 $P = 0$，转入缺页处理，当前查找停止。
- **形成物理地址后查 Cache：**用物理地址选行，再同时检查有效位和标记。$V = 1$ 且标记相同才命中；否则从主存调块，按块内偏移取出目标字节。

读取 03D4H：TLB 命中，Cache 也命中。虚页号为 0FH，页内偏移为 14H。VPN 的低 2 位给出 TLB 组号 3，其余位给出标记 03H。表 A 的组 3、路 2 有效且标记为 03H，取得页框号 0DH，无需再查页表。物理地址为

$$0\mathrm{DH} \times 40\mathrm{H} + 14\mathrm{H} = 354\mathrm{H}.$$

拆分 354H，得到 Cache 标记 0DH、行号 5H、块内偏移 0。表 C 中第 5H 行的 $V=1$，标记也是 0DH，因此从该行偏移 0 的位置读得 **36H**。

读取 00F1H：TLB 未命中，页面已驻留，Cache 未命中。虚页号为 03H，页内偏移为 31H；TLB 组号为 3、标记为 00H。组 3 没有有效的匹配项，因此查表 B。虚页 03H 的 $P=1$、页框号为 02H，说明页面已在主存，**不发生缺页**。物理地址为

$$02\text{H}\times 40\text{H}+31\text{H}=0\text{B1H}.$$

拆分 0B1H，得到 Cache 标记 02H、行号 CH、块内偏移 1。第 CH 行虽保留着相同标记，但 $V=0$，所以仍未命中。须从主存调入地址 0B0H 至 0B3H 所在的 4 B 块，再取偏移 1 的字节。题中没有给出该主存字节的内容，**只能确定取数路径，不能确定字节值**。

读取 0229H：TLB 未命中，随后发生缺页。虚页号为 08H，页内偏移为 29H；TLB 组号为 0、标记为 02H。组 0 没有有效的匹配项，转查页表。表 B 中虚页 08H 的 $P=0$，因此触发缺页异常。此时尚无可用页框号，**不形成物理地址，也不继续查数据 Cache**。由操作系统完成缺页处理、更新映射并重新执行该访问后，才继续取数。

三次访问分别在 TLB、页表和 Cache 处作出不同判断。TLB 缓存的是映射，页表存在位说明页面是否驻留，Cache 保存的是数据块：**TLB 未命中不等于缺页，页面已驻留不等于 Cache 命中，标记相同也不能代替有效位检查**。真实系统还须区分非法地址和访问越权；本题通过 “地址合法、允许读” 的条件将这两类异常排除。

课程连接·从路径列事件

先列访问哪些层级、命中或缺失时发生哪些访问，再按模型计算平均时间；不要把不同路径的额外时间重复计入。

3.15 先画访问路径，再写时间与容量

存储系统综合题的顺序

1. 写明地址位数、按字节还是按字编址、块与页大小，把地址先拆成有意义的字段。
2. 按题设画 TLB、页表、Cache 和主存的实际查找路径；分清并行与串行，缺页与普通缺失。
3. 每一步同时检查标记、有效或存在位、权限；得到物理地址后再按指定 Cache 模型查数据。
4. 时间式以完整事件分支加权；容量式先分数据位、标记和有效/脏/替换等元数据。
5. 回代一个边界：空 Cache、组数 1、路数 1、页面不驻留或跨页，查看路径是否仍一致。

3.15.1 地址拆分为什么不遗漏信息

从整除和余数恢复地址

设字节地址 A，块大小 B，组数 G，各量为适用范围内的整数。先作 $A=qB+d$，其中 $0\le d<B$；再作 $q=tG+s$，其中 $0\le s<G$。因此 $A=(tG+s)B+d$，恰由标记 t、组号 s 和块内偏移 d 恢复。B、G 为 2 的幂时，这两次除法正对应二进制位段切分；不是 2 的幂时仍可整除取模，但不能硬套整数位数。

□

3.15.2 主存器件分类与刷新开销

不变量·相联程度不直接等于延迟

全相联能并行比较多行，组相联只比较一个组；实际延迟还受比较器、布线和编码影响。固定实现才可比较快慢。映射限制候选位置，替换选择牺牲行，电路时序决定访问时间。

只读存储器的“只读”描述其正常使用或修改方式，不表示所有 ROM 永远无法改写。掩膜 ROM 由制造过程固定，PROM 可一次编程，EPROM 可通过特定方式擦除重写，EEPROM 支持电擦除；Flash 也是电可擦写的非易失存储，但擦除粒度通常较大。判断题先区分是否掉电保持、能否改写、擦写粒度与速度，避免把这些维度混成同一个分类。

刷新保持 DRAM 电容状态，读后恢复则由一次读出带来的状态变化引起；二者目的相关，但不能据“读后要恢复”推断只有被访问的行才需刷新。若 4096 行必须在 64ms 内各刷新一次，平均每 15.625 微秒需安排一行刷新；若每次刷新占 100ns，则理想均匀安排下占用比例约 0.64%。这是自拟预算示例；具体实现可能一条命令刷新多行或由芯片内部完成，要服从题设。

3.15.3 相联查找的并行性与延迟条件

相联映射描述能放在哪里以及怎样匹配标记，不直接决定所有比较的实际电路延迟。全相联可以并行比较多行标记，组相联可以只比较目标组中的若干行；比较器数、扇入、布线和编码都会影响时间。比较“最快”“最慢”时须固定具体实现，不能仅用相联度高低给出普遍时延定理。

写策略同样分成两次决策：写命中采用直写或回写，写缺失采用分配或不分配。回写的英文为 write-back，写分配为 write-allocate；二者分别处理命中更新和缺失装入，名字不能互换。脏块替换时才需写回，并非每次替换都必然产生一次主存写。

3.16 存储器分类、刷新与双端口访问

按存取方式，随机存取 RAM 可按任意地址访问，通常不因地址位置不同而有明显不同的寻址步骤；顺序存取 SAM 必须沿记录次序定位，如磁带；直接存取 DAM 先选大区域再在局部定位，如磁盘；相联存取按内容或标记比较选择信息。随机存取描述访问方式，与“断电是否保持”是两个分类维度。

SRAM 利用双稳态电路保存位，读取通常非破坏性，无需周期刷新；DRAM 利用电容电荷保存位，读出后要再生，并需在刷新周期内刷新所有行。读后再生恢复刚读出的电荷，定时刷新防止未读单元因漏电丢失信息，二者目的不同。DRAM 按行刷新时，刷新地址计数与行数有关，而不取决于存储器容量中的每一个字。

集中刷新在固定时段连续刷新全部行，形成较长死时间；分散刷新把正常存取和刷新交替安排，增加每次存取周期；异步刷新将刷新动作均匀分散在保持时间内。设有 128 行、每行刷新一次需 0.5 μs、要求 2 ms 内全部刷新，则相邻行刷新启动间隔可按 $2\text{ms}/128 = 15.625\,\mu\text{s}$ 安排，总刷新时间为 64 μs，占比 3.2%。

双端口 RAM 具有两套独立地址、数据和控制线路，两端可并行访问不同单元。同一单元上的读写或写写冲突，需要通过忙信号、仲裁或芯片规定的访问规则处理；“双端口”并不自动保证任何两次访问都无冲突。

3.16.1 ROM 各类别的可改写能力

课程连接·页与擦除块

闪存按页读取或编程，按块擦除；逻辑覆盖常转为异地写入并更新映射，磨损均衡不能消除擦除次数上限。

器件	内容写入或修改方式	需要注意的条件
掩膜 ROM	制造时写入	使用阶段不能修改
PROM	用户一次编程	写入后通常不能重新编程
EPROM	紫外线擦除后重写	通常整片擦除，修改不便
EEPROM	电擦除、电改写	改写速度与次数受器件限制
Flash	按块擦除、按规定粒度写入	写前擦除，读写和擦除粒度不同

3.17 SSD 的页、块与磨损均衡

SSD 由闪存翻译层和多个闪存芯片组成，芯片内再分块，块内再分页。系统给出逻辑块号，闪存翻译层 FTL 把它映射为物理页地址。NAND Flash 通常按页读取或编程、按块擦除；页与块大小取决于具体器件，课件例示页约 512 B～4 KiB、块约 16～512 KiB，这些是示例范围，不能作为所有 SSD 的固定参数。

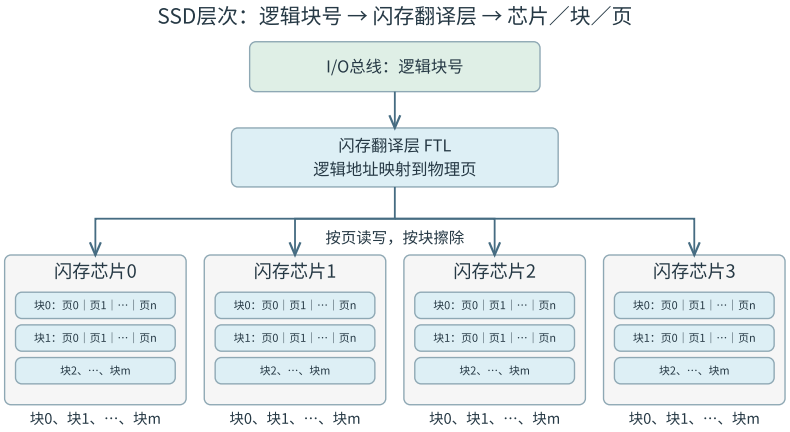


Figure 3.4: 系统逻辑块号经 FTL 映射到闪存芯片中的块与页

旧页中的数据需要更新时，控制器可把有效数据和新内容写入另一个已擦除块，更新映射，再把旧页标为失效。垃圾回收把仍有效的页集中搬走后擦除旧块，恢复空闲空间；额外搬移会产生写放大。读写快、无机械寻道、低噪声和抗震是相对机械磁盘的典型特点；擦写寿命、容量成本和写入管理是另一组比较维度。

磨损均衡通过分散擦写次数延长整体寿命。**动态磨损均衡**在发生写入时，优先使用擦除次数较少的空闲块；长期不改写的冷数据可能一直占着某些块。**静态磨损均衡**还会主动搬移冷数据，让擦写较少的块参加新写入，使各块使用更均匀。主动搬移增加了管理开销，不能把静态均衡理解为 “完全不移动数据”。

3.18 位扩展、字扩展与片选译码

课程连接·位扩展与字扩展

位扩展让多片提供同一地址的不同数据位；字扩展选择不同芯片提供更多地址，两者可组合。

芯片标称 $N \times w$ 表示 N 个单元、每单元 w 位。用 $n \times b$ 芯片组成 $N \times W$ 存储器，若两维均为整数倍，片数为 $(N/n)(W/b)$。

位扩展让若干芯片同时接收相同地址，各自负责不同数据位，共同构成更宽的存储字；片选相同，数据线分组连接。**字扩展**让各组芯片分别负责不同地址区间，低位地址接到组内所有芯片，高位地址经译码产生互斥片选。字位同时扩展时，先确定同组并行芯片数，再确定地址组数。

3.18.1 例：用 16K×8 位芯片组成 32K×16 位

目标字数扩大 2 倍，字长扩大 2 倍，共需 4 片。每组 2 片分别接 $D_7 \sim D_0$ 和 $D_{15} \sim D_8$；$A_{13} \sim A_0$ 并联到四片，A_{14} 选择两组。按字编址时，组 0 范围 0000H～3FFFH，组 1 范围 4000H～7FFFH。

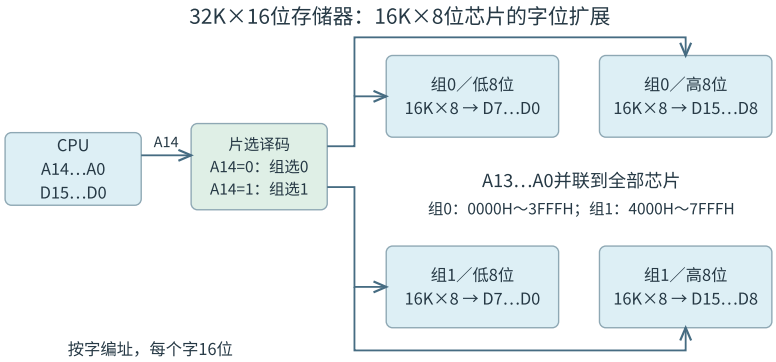


Figure 3.5: 同组两片作位扩展，A14 选择两组作字扩展

若按字节编址、每字 16 位，还需要说明最低地址位怎样区分字节以及字节使能信号如何连接，不能直接把上述按字地址范围照搬。片选有效电平由芯片和译码器图中的小圆圈、横线约定决定；逻辑表达式必须同时考虑访存使能和地址译码，避免 I/O 访问错误选中内存。

全译码使用全部需要判别的高位地址，地址对应关系唯一；部分译码省略某些高位，可能让多个地址选择同一物理单元，形成地址重叠；线选法直接用地址线选片，电路简单，但可用地址组合与地址连续性受限制。

3.19 交叉编址的连续访问时间

高位交叉用地址高位选择模块，同一连续区间常落在同一模块，主要扩容。低位交叉用低位选择模块，连续地址轮流分布到各模块，允许各模块启动时间交错。

设每个模块访问周期为 T，总线传送一个字需 τ，模块数 m 满足 $m\tau \geq T$，按连续地址访问且无其他冲突时，可每隔 τ 启动一个新字。读取连续 n 字的理想时间为 $T + (n-1)\tau$。若 $m\tau < T$，最先使用的模块尚未完成恢复便又被访问，须加入等待。

例如 $T = 200$ ns、$\tau = 50$ ns，至少 4 个模块才可连续交错启动；读取 8 字需 $200 + 7 \times 50 = 550$ ns。步长访问还要看模块号序列：模块数 4 且步长 4 时，总是访问同一模块，即使地址不同也不能获得连续逐字带宽。

3.20 Cache 替换：把命中后的状态也写出来

课程连接·命中也会改变 LRU

LRU 命中后需把命中块更新为最近使用；FIFO 命中一般不改变装入顺序。每次访问后写出状态，避免只数缺失。

直接映射的候选位置唯一，冲突时无需在多行中选择牺牲者；组相联只在所选组内替换，全相联在全部行内替换。有效位为 0 的空行优先使用。替换策略独立于写策略；被替换行若采用写回且脏位为 1，还要先回写。

FIFO 按装入时间淘汰最早装入行，命中不改变装入顺序；LRU 按最近使用情况淘汰最久未使用行，命中也要更新次序；LFU 按累计访问次数选择，需规定计数初值及并列时的规则；随机替换由硬件产生选择，成本较低。

4 行全相联 Cache 初始为空，访问主存块 $1,2,3,4,1,2,5,1,2,3,4,5$：

算法	命中位置（从 1 计数）	命中数／缺失数
FIFO	第 5、6 次	2 ／ 10
LRU	第 5、6、8、9 次	4 ／ 8

本序列中 LRU 命中 4 次、FIFO 命中 2 次，牺牲块和中间内容也不同。例如访问 5 前，FIFO 仍先淘汰 1，LRU 先淘汰 3；不能只核对最终命中率，就认为过程表正确。实现 LRU 可用访问次序栈或相应状态编码；题目采用计数器法时还须遵守其更新规则。

3.21 平均访问时间的两个模型

设 Cache 访问时间为 t_c、主存时间为 t_m、命中率为 h。若 Cache 和主存同时开始访问、命中时用 Cache 结果，则平均时间按 $ht_c+(1-h)t_m$ 计算。若先访问 Cache，确认未命中后再启动主存，则为 $t_c+(1-h)t_m$。还存在题目直接给出 “缺失损失” 的形式 $t_c+(1-h)p$，其中 p 须按定义判定是否含取块、回写和重新访问。

课件模型取 $t_m=5t_c$、$h=0.95$：同时访问时平均 $1.2t_c$，相对仅访问主存加速 $5/1.2\approx4.17$ 倍；先 Cache 后主存时平均 $1.25t_c$，加速 4 倍。这里的同时访问是计算模型，不是所有处理器的必然实现。

写直达在写命中时同时更新下层，写缓冲可以暂存待写内容；写回只更新 Cache 并置脏位，替换脏行时才写下层。写分配与非写分配处理的是写缺失：前者调入块后写，后者绕过当前 Cache 写入下层。常见组合属于工程选择，分析题目时按明确给出的策略推演。

3.22 磁盘记录与 RAID 的边界

磁盘由盘面、磁道、扇区组成，柱面是不同盘面上半径相同的磁道集合。一次访问通常包含寻道、旋转等待和数据传输；平均旋转等待在随机到达且均匀假设下为半圈。容量按有效盘面数 $\times$ 每面磁道数 $\times$ 每道扇区数 $\times$ 每扇区字节数计算，格式开销和保留区域需按题设处理。

RAID0 条带化提升并行带宽而无冗余；RAID1 镜像提高冗余但有效容量约减半；RAID3、RAID4 采用专用校验组织，RAID5 分布式校验避免固定校验盘成为所有写入的同一热点。冗余的容错范围与级别、故障数量有关，不能据 “有校验” 就断言任意多盘损坏都能恢复。写入更新还可能产生数据与校验的一致性开销。

3.22.1 计算与辨析自查

1. 四模块低位交叉中连续字和步长 4 访问的模块分布有何不同？2. 32K×16 位目标由 16K×8 位芯片组成需要几片？3. 为什么 FIFO 命中后不改队列而 LRU 要改？4. SSD 按页写、按块擦除会引出哪些管理动作？

课程连接·地址划分顺序

依题意先确定块内偏移，再根据组数确定组号；比较标记前还要验证有效位。

答案。1. 连续字轮流访问 0、1、2、3 号模块，步长 4 始终访问同一模块。2. 4 片，两片一组、两组字扩展。3. FIFO 记录装入时间，LRU 记录使用时间。4. 异地更新、地址映射、失效页管理、垃圾回收和磨损均衡。

3.23 主动回忆自测

3.23.1 配套自测

1. 原创补充：32 位字节地址、Cache 数据区 4 KB、块 16 B、2 路组相联，求偏移、组号、标记位数。假定另需有效位和修改位，先不计替换位，每行多少位？
2. 原创补充：两路某组按 LRU 访问 A、B、A、C、B，初始空，列每次命中情况与最近到最久次序。
3. 【配套例题】下列事件中，不是在 MMU 地址转换过程中检测的是（ ）。A. 访问越权　B. Cache 缺失　C. 页面缺失　D. TLB 缺失。

答案。1. 4、7、21 位；每行 $128+21+1+1=151$ 位。2. 缺失 [A]、缺失 [B,A]、命中 [A,B]、缺失 [C,A]、缺失 [B,C]。3. B。MMU 负责地址转换和相关保护；Cache 缺失属于 Cache 查找结果，不能与 TLB 缺失等同。

3.23.2 遮住答案复述知识关系

主动回忆·自拟

4. 自拟：块大小 64B、物理地址 32 位、Cache 共 16 行，采用 2 路组相联。组号多少位？与直接映射相比，标记位数怎样变化？
5. 自拟：一次查 Cache 为 2ns，命中率 95%，缺失后额外访问下一层 80ns，AMAT 是多少？如果“缺失时间 80ns”已经含查 Cache，公式又怎样写？
6. 自拟：4 体低位交叉、按字编址，字地址 0、4 连续访问为何可能冲突？是否任何两次同模块访问必冲突？
7. 自拟：TLB 未命中、页表存在位为 1、Cache 有效位为 0 时，分别发生哪两种缺失？是否缺页？
8. 自拟：回写 Cache 的一行被淘汰，是否必定写主存？还要看哪一位？
9. 自拟：DRAM 无需掉电也可能丢信息，Flash 掉电仍保留信息但为什么不能按 SRAM 方式任意覆盖写？

3.23.3 参考答案

自拟题核对答案

4. 8 组、组号 3 位，偏移 6 位，标记 23 位；直接映射 16 行，行号 4 位、标记 22 位，因此 2 路相联标记多 1 位。
5. 额外 80ns 时 $2 + 0.05 \times 80 = 6$ns；已含查 Cache 则 $0.95 \times 2 + 0.05 \times 80 = 5.9$ns。
6. 两者模 4 都为 0，若第二次请求到达时模块尚忙则冲突；若时间间隔足够则不冲突。
7. TLB 缺失和 Cache 缺失；页面已驻留，不发生缺页，仍需正常读取物理主存。
8. 不一定；只有脏位表明内容已修改且需要回写时才写回，干净副本可直接丢弃。
9. DRAM 电荷泄漏，需要周期刷新；Flash 编程和擦除粒度、允许的位变化及寿命条件不同，通常需先擦后写并由控制器管理。

课程连接·地址划分顺序

依题意先确定块内偏移，再根据组数确定组号；比较标记前还要验证有效位。